

Electronic Products: In-the-Loop Testing Aids Embedded System Validation



Jay Abraham is manager of product marketing for verification and validation products at MathWorks. He has over 20 years of software and hardware design experience

Embedded systems software can provide significant competitive advantage to systems. For example, sophisticated embedded software in automotive applications helps improve safety, comfort, functionality, fuel economy and vehicle performance. Implementing functionality in software versus hardware decreases manufacturing cost. Further, software enables reuse and can be updated more frequently to satisfy customer needs.

With increasing applications of embedded software, complexity and sophistication of software algorithms continue to increase. As a result, software becomes more difficult to design, implement, test and verify. With traditional software development processes, complex verification and validation overhead, and late defect detection, puts the benefits of adding sophisticated software to embedded applications at risk.

As a way to deal with increased complexity, the embedded software development industry recognised and embraced model based design (MBD) based on graphical modelling and simulation. These techniques provide an opportunity to improve verification and validation processes.

In the early design stages, people can develop behavioural models based on textual requirements. They can clarify and de-

fine detailed low-level requirements. Such models may have the basic architecture of the solution but these are independent of the target platform. These target-independent behavioural models are used for design verification and early requirements validation. A model that is used to capture key requirements, demonstrate correct behaviour in simulation and traceability to high-level requirements is referred to as an executable specification.

Further refinement of the executable specification and addition of implementation details (that is, to produce source codes) result in a model that is ready for final implementation. Often, such a model is optimised for code generation, that is, the process of automatically generating production C or C++ codes for compilation and execution on a target-embedded processor. It honours data types, target architecture and the required coding style.

It is important that the behaviour and execution of the simulation model, that is, executable specification, match the behaviour and execution of generated production codes.

In this article, we refer to testing that determines the correct behaviour of the implementation model and of generated codes as code verification. With graphical modelling and simulation, verification and testing tasks can be performed earlier in the development process. These tasks include:

- Model testing in simulation
- Software-in-the-loop testing
- Processor-in-the-loop testing
- Hardware-in-the-loop testing

Model testing in simulation

Unlike static paper designs, an executable specification can be evaluated in simulation. Typically, this is done by changing model parameters, input signals and reviewing outputs and responses in a simu-

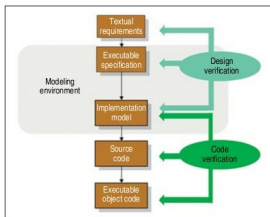


Fig. 1: Overview of verification with models